UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

APUNTES

Sistemas Distribuidos

Clase 12 – Confiabilidad operativa: objetivos, experimentos e incidentes

Departamento de Computación

Ingeniería Informática – Comisión A

Pilar – Segundo cuatrimestre 2026

Equipo docente

2026

1 Introducción

La clase 10 mostró cómo *ver* lo que ocurre dentro de un sistema distribuido; la clase 11, cómo decidir de antemano de qué manera *falla*. Queda una pregunta que ninguna de las dos responde por sí sola: ¿cuándo está el sistema funcionando *suficientemente bien*? Sin una respuesta explícita, cada discusión operativa —si se libera una versión nueva, si una alerta merece despertar a alguien, si un *circuit breaker* bien configurado alcanza, si un incidente fue grave— se resuelve por intuición, por jerarquía o por quién argumenta mejor.

Esta clase reúne las prácticas con las que los equipos que operan sistemas en producción convierten esa pregunta en decisiones verificables. Son seis temas:

1. **Indicadores, objetivos y acuerdos (SLI, SLO y SLA):** cómo definir con números qué significa que el servicio funcione suficientemente bien (sección 2).
2. **Error budget:** cuánta falla se tolera y cómo se usa esa tolerancia para tomar decisiones (sección 3).
3. **Alertas sobre el objetivo (*burn rate*):** cómo avisar a tiempo de que el objetivo está en peligro sin llenar al equipo de falsas alarmas (sección 4).
4. **Experimentos bajo falla (*Chaos Engineering*):** cómo comprobar, antes de que ocurra un incidente real, que el sistema se comporta como se supone cuando algo falla (sección 5).
5. **Gestión de incidentes:** cómo responder de forma coordinada cuando algo sale mal (sección 6).
6. **Postmortems:** cómo aprender de cada incidente para que no se repita (sección 7).

Antes de entrar en cada tema, esta introducción presenta la idea que los une —la confiabilidad como gestión de riesgo—, cómo se encadenan entre sí y el sistema que se usa como ejemplo a lo largo de todo el apunte.

1.1 La confiabilidad como gestión de riesgo

La intuición inicial suele ser que el objetivo es que el sistema no falle nunca. Sin embargo, pasado cierto punto, aumentar la confiabilidad es peor para el servicio y para sus usuarios.[1] Hay tres razones.

- **El costo no es lineal.** Cada mejora incremental de confiabilidad puede costar cien veces más que la anterior. El costo tiene dos dimensiones: los recursos redundantes (máquinas, réplicas, capacidad ociosa) y el *costo de oportunidad*: los ingenieros que trabajan en reducir el riesgo no construyen funcionalidades que el usuario ve.
- **El usuario no percibe la diferencia.** La experiencia del usuario está dominada por los componentes menos confiables de la cadena, como la red celular o su propio dispositivo: quien usa un teléfono confiable al 99 % no distingue un servicio confiable al 99,99 % de uno al 99,999 %.
- **La confiabilidad compite con la velocidad de cambio.** Maximizar la estabilidad limita cuán rápido se desarrollan y entregan funcionalidades nuevas.

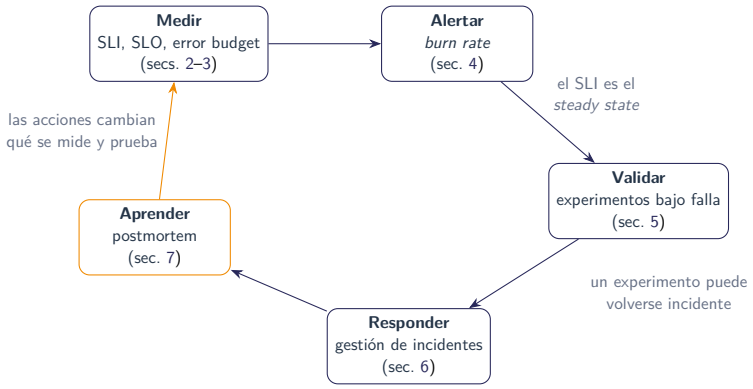
Por eso conviene tratar el riesgo como un continuo y ubicar cada servicio en el punto adecuado: suficientemente confiable, pero no más de lo necesario. El objetivo de disponibilidad funciona entonces a la vez como mínimo y como máximo.[2] Que sea un mínimo es lo esperable: por debajo, los usuarios sufren. Que también sea un máximo es lo contraintuitivo. Si un servicio con un objetivo de 99,99 %

[1] Beyer et al. (2016), cap. 3.
[2] Beyer et al. (2016), cap. 3.

logra en la práctica 99,999 %, nadie nota ese nueve adicional, y el esfuerzo que costó podría haberse dedicado a funcionalidades nuevas, a pagar deuda técnica o a bajar los costos de operación.

1.2 Cómo se encadenan los temas

Los seis temas no son prácticas independientes: forman un ciclo. Medir (SLI, SLO y error budget) da una definición numérica de lo que es normal. Alertar vigila esa definición. Validar la usa como punto de comparación en los experimentos. Responder la usa para decidir cuándo terminó un incidente. Y aprender produce acciones que cambian qué se mide, qué se alerta y qué se prueba.

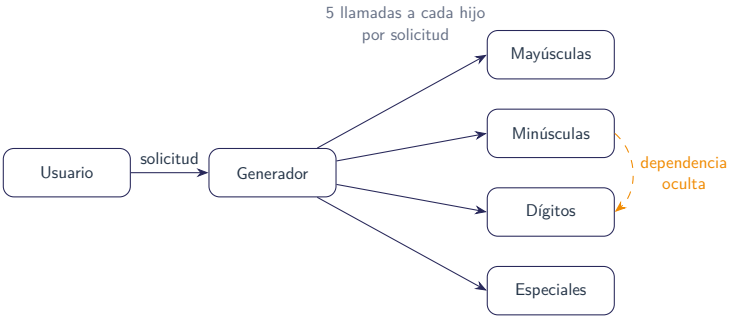


El orden de la clase sigue el del ciclo, y no es arbitrario: el experimento bajo falla de la sección 5 usa el SLI como definición de comportamiento normal y el *burn rate* como criterio para abortar, de modo que medir y alertar tienen que venir primero. La actitud general se resume en una frase: la esperanza no es una estrategia.[3] Ni la confiabilidad, ni la eficacia de un mecanismo de resiliencia, ni la capacidad de un equipo para responder a un incidente se suponen: se miden, se prueban y se ejercitan.

1.3 El ejemplo recurrente: el generador de contraseñas

Para que los conceptos no queden abstractos, todo el apunte usa el mismo sistema: el *generador de contraseñas* de la clase 11. Sobre él se definen sus indicadores y objetivos (sección 2), se calculan su error budget (sección 3) y sus alertas (sección 4), se diseñan experimentos (sección 5), se escribe su runbook (sección 6) y se analiza un incidente (sección 7).

Qué hace. Un usuario —una persona en una página de registro, u otra aplicación— envía una *solicitud de generación de contraseña* y recibe una contraseña que cumple una política, por ejemplo: incluir mayúsculas, minúsculas, dígitos y caracteres especiales. Para armarla, el servicio generador llama varias veces a cuatro servicios hijos, uno por cada tipo de carácter: *mayúsculas*, *minúsculas*, *dígitos* y *especiales*. A lo largo del apunte, "solicitud" y "pedido" se usan como sinónimos.



[3] Beyer et al. (2016), cap. 3: "Hope is not a strategy", lema informal de los equipos de SRE de Google.

Qué conviene recordar de la clase 11.

- Cada solicitud hace 5 llamadas a cada servicio hijo, es decir, 20 llamadas en total. Cualquier falla por llamada se multiplica por solicitud.
- El servicio de *minúsculas* consulta internamente al de *dígitos*, una dependencia que no figura en el diagrama de arquitectura original.
- Omitir los caracteres especiales cuando ese servicio no responde no es una degradación aceptable: produce una contraseña que puede violar la política del usuario.
- El generador tiene *timeouts*, reintentos y un *circuit breaker* por dependencia. Esta clase pregunta si eso alcanza, y cómo saberlo.

Supuestos para los cálculos. Se supone un tráfico promedio de 5 solicitudes por segundo: 18 000 por hora y unos 12,96 millones en 30 días ($5 \times 60 \times 60 \times 24 \times 30$).

Qué significa, para este servicio, funcionar bien. Desde el punto de vista de quien lo usa, son dos cosas: recibir una contraseña válida y recibirla rápido. Esas dos frases, todavía vagas, son el punto de partida de la sección siguiente.

2 Medir: indicadores, objetivos y acuerdos

Todo sistema existe para darle un servicio a alguien. El generador da contraseñas a quienes se registran en una aplicación; una base de datos atiende lecturas y escrituras de otros servicios; un *pipeline* entrega datos procesados a un tablero. Cada servicio lleva una promesa implícita sobre cómo se comporta: que responda, que lo haga rápido, que la respuesta sea correcta. Mientras esa promesa quede implícita, cada persona la interpreta a su manera. Para quien desarrolla, el servicio anda bien si no hay errores en los logs; para quien lo usa, si la página de registro no se traba; para quien lo vende, si nadie se queja.

Medir sirve para convertir esa promesa en un acuerdo explícito sobre qué significa que el servicio funcione *suficientemente bien*. El acuerdo tiene tres piezas: qué se mide (el indicador), qué valor se considera aceptable (el objetivo) y qué pasa si no se cumple (el acuerdo con consecuencias). Sin la primera no hay datos; sin la segunda, los datos no permiten decidir nada; sin la tercera, frente a un cliente que pagó por el servicio, el objetivo queda como una expresión de deseo. En definitiva, es imposible operar bien un servicio sin entender qué comportamientos importan y cómo medirlos.[4]

2.1 Tres términos que se confunden

El vocabulario estándar distingue tres conceptos:[5]

SLI (*service level indicator*): una medida cuantitativa, cuidadosamente definida, de algún aspecto del nivel de servicio. Ejemplos: la latencia de las solicitudes, la tasa de error, el *throughput*, la disponibilidad o, en sistemas de almacenamiento, la durabilidad.

SLO (*service level objective*): un valor objetivo, o un rango de valores, para un nivel de servicio medido por un SLI. Su forma natural es "SLI $\leq$ objetivo" o "límite inferior $\leq$ SLI $\leq$ límite superior".

[4] Beyer et al. (2016), cap. 4.
[5] Beyer et al. (2016), cap. 4.

SLA (*service level agreement*): un contrato, explícito o implícito, con los usuarios, que incluye las consecuencias de cumplir o no los SLOs que contiene.

Para el generador, un ejemplo de cada uno (los valores se justifican en la sección 2.5):

- **SLI:** el porcentaje de solicitudes que reciben una contraseña válida.
- **SLO:** ese porcentaje es al menos 99,9 % en los últimos 30 días.
- **SLA:** si en un mes calendario el porcentaje queda por debajo de 99,5 %, el cliente recibe un descuento en la factura (ejemplo ilustrativo).

Las secciones 2.2, 2.3 y 2.4 desarrollan cada término.

2.2 SLI: el indicador

2.2.1 El SLI como proporción de eventos buenos

Conviene expresar todo SLI como un cociente:[6]

$$\text{SLI} = \frac{\text{eventos buenos}}{\text{eventos válidos}} \times 100\,\%$$

La forma de cociente tiene dos ventajas. Primero, todos los SLIs quedan en la misma escala, de 0 % a 100 %, donde 100 % es lo mejor, lo que simplifica los objetivos y el cálculo del error budget. Segundo, mide bien las fallas *parciales*, que en un sistema distribuido son la norma.

La palabra *válidos* no es un detalle: define la población. Una solicitud mal formada que el servicio rechaza con un error 4xx no dice nada sobre la salud del servicio y normalmente se excluye, del mismo modo que en la clase 11 el *circuit breaker* no contaba los 4xx como fallas de la dependencia.

Disponibilidad. La clase 8 distinguió el *uptime*, la fracción del tiempo en que un proceso estuvo levantado y alcanzable, de la disponibilidad semántica, la fracción de intentos válidos que cumplen la promesa de negocio. Un SLI de disponibilidad mide la segunda:

$$\text{disponibilidad} = \frac{\text{solicitudes exitosas}}{\text{solicitudes válidas}}$$

Así cuenta también las fallas parciales: un proceso puede estar levantado, y responder a un chequeo de salud (*health check*), mientras una parte de las solicitudes falla.[7]

Latencia: un percentil o una proporción. Hay dos formas, que la sección 2.2.4 compara:

- "el p99 de la latencia es menor que 300 ms", evaluado en cada ventana de 5 minutos. Es la forma habitual en los *dashboards*.
- "la proporción de solicitudes respondidas en menos de 300 ms". Si en una ventana hubo 1 500 solicitudes y 1 488 tardaron menos de 300 ms, el SLI vale $1\,488/1\,500 = 99{,}2\,\%$.

Otros ejemplos.

- **Tasa de error:** la fracción de solicitudes que fallan; es el complemento de la disponibilidad por solicitudes.

[6] Beyer et al. (2018), cap. 2.
[7] Beyer et al. (2016), cap. 3.

- ***Throughput*:** solicitudes atendidas por segundo. Sirve como indicador, pero rara vez como objetivo propio, porque la cantidad de solicitudes la deciden los usuarios, no el servicio.[8]
- **Frescura:** en un *pipeline*, la proporción de registros actualizados hace menos de 10 minutos.
- **Durabilidad:** en almacenamiento, la proporción de registros escritos que pueden volver a leerse.[9]

2.2.2 Especificación e implementación

Conviene separar dos niveles:[10]

Especificación: lo que se cree que le importa al usuario, con independencia de cómo se mida. Por ejemplo: "la proporción de solicitudes de generación de contraseña que reciben una contraseña válida".

Implementación: la especificación más una forma concreta de medirla. Una misma especificación admite varias implementaciones, con distintos compromisos entre calidad, cobertura y costo.

Para el generador, la misma especificación puede implementarse así:

Implementación	Ventaja	Limitación
Logs del servicio generador	Barata y disponible desde el primer día.	No ve las solicitudes que nunca llegaron al proceso (el proceso caído, la red cortada antes).
Métricas del balanceador	Más cerca del usuario; ve las solicitudes que el servicio no atendió.	No sabe si la contraseña devuelta cumple la política.
Cliente sintético (*prober*)	Mide el recorrido completo y puede validar el contenido.	Mide tráfico artificial, no el de los usuarios reales.
Instrumentación en el cliente	Es lo más cercano a la experiencia real.	Más costosa; mezcla problemas propios con los de la red del usuario.

Como regla general, conviene medir lo más cerca del usuario que se pueda,[11] porque medir solo en el servidor puede pasar por alto problemas que afectan al usuario: un servidor puede responder rápido mientras la página tarda en volverse usable por un problema del JavaScript que corre en el navegador.[12] Para un primer SLI, sin embargo, conviene elegir una implementación que requiera poco trabajo de ingeniería y refinarla después.

2.2.3 Qué medir según el tipo de servicio

No toda métrica disponible debe ser un SLI. Elegir demasiados indicadores dispersa la atención; elegir muy pocos deja comportamientos importantes sin examinar. Un puñado de indicadores representativos suele alcanzar, y el tipo de servicio sugiere cuáles:[13]

[8] Beyer et al. (2016), cap. 4.
[9] Beyer et al. (2018), cap. 2.
[10] Beyer et al. (2018), cap. 2.
[11] Beyer et al. (2018), cap. 2.
[12] Beyer et al. (2016), cap. 4.
[13] Beyer et al. (2016), cap. 4; Beyer et al. (2018), cap. 2.

Tipo de servicio	SLIs habituales	Pregunta que responden
Atiende solicitudes de usuarios	Disponibilidad, latencia, calidad	¿Se respondió? ¿Cuánto tardó? ¿La respuesta estaba degradada?
Almacenamiento	Latencia, disponibilidad, durabilidad	¿Cuánto tarda leer o escribir? ¿Los datos siguen ahí cuando se los necesita?
Pipeline de datos	Frescura, corrección, cobertura, *throughput*	¿Cuán viejos son los datos? ¿Cuántos se procesaron, y bien?
Cualquier sistema	Corrección	¿Se devolvió la respuesta correcta?

La corrección entra en el SLI. El generador muestra por qué la corrección no es opcional. Un SLI de disponibilidad que contara como "buena" toda solicitud respondida con un código 200 marcaría como éxito una contraseña sin caracteres especiales devuelta por un *fallback* mal decidido. El evento bueno tiene que ser "respuesta 200 *con una contraseña que cumple la política*". Lo que la clase 11 decidió como regla de producto (no degradar quitando los especiales) se refleja acá en la definición del indicador.

2.2.4 Latencia: distribuciones, no promedios

La clase 10 presentó la falacia del promedio; en los SLIs tiene consecuencias directas. Incluso una métrica que parece simple agrega datos: un sistema que atiende 200 solicitudes por segundo en los segundos pares y ninguna en los impares tiene el mismo promedio que uno que atiende 100 por segundo de forma constante, pero su carga instantánea es el doble.[14] Con la latencia ocurre lo mismo: puede haber un sistema en el que la solicitud típica tarda unos 50 ms pero el 5 % tarda veinte veces más, un cambio en la cola que un monitoreo basado en el promedio no muestra. Los percentiles altos (p99, p99,9) muestran un peor caso plausible; la mediana, el caso típico. Además, los usuarios suelen preferir un sistema algo más lento a uno con mucha varianza.

Las dos formas de escribir un SLI de latencia de la sección 2.2 no son equivalentes:[15]

- **Basada en tiempo** ("el p99 es menor que 300 ms en cada ventana de 5 minutos"): cada ventana entera queda marcada como buena o mala.
- **Basada en eventos** ("la proporción de solicitudes respondidas en menos de 300 ms"): cada solicitud es buena o mala.

La segunda tiene la forma de cociente y es más precisa; la sección 4 muestra por qué importa al calcular el error budget.

2.2.5 Estandarizar las definiciones

Un SLI queda ambiguo si no se fija cómo se mide. Conviene acordar *templates* comunes que fijen: el intervalo de agregación ("promediado en 1 minuto"), la región de agregación ("todas las tareas de un clúster"), la frecuencia de medición, qué solicitudes se incluyen, cómo se obtiene el dato ("en el servidor", "en el balanceador") y, para la latencia, qué se mide exactamente (por ejemplo, el tiempo hasta el último byte).[16] Con *templates* compartidos, cada SLI nuevo solo declara lo que difiere del estándar, y todos lo leen igual.

[14] Beyer et al. (2016), cap. 4.
[15] Majors et al. (2022), cap. 12.
[16] Beyer et al. (2016), cap. 4.

2.3 SLO: el objetivo

2.3.1 Cómo se escribe

Un SLO bien escrito dice cómo se mide y bajo qué condiciones vale. Por ejemplo: "el 99 % de las llamadas Get termina en menos de 100 ms, medido en todos los servidores del *backend*". Si importa la forma de la distribución, se pueden fijar varios umbrales a la vez (90 % en menos de 1 ms, 99 % en menos de 10 ms, 99,9 % en menos de 100 ms); si hay clases de clientes con necesidades distintas, objetivos separados para cada una.[17]

2.3.2 Cómo elegir el valor

Conviene empezar por lo que le importa al usuario, no por lo que es fácil de medir: partir de lo fácil produce SLOs poco útiles, y a veces funciona mejor ir desde el objetivo deseado hacia el indicador. Algunas reglas para elegir:[18]

No elegir el objetivo a partir del desempeño actual. Adoptar sin reflexión lo que el sistema logra hoy puede atar al equipo a sostener con esfuerzos heroicos algo que no mejora sin rediseño.

Mantenerlo simple. Un SLI que combina varias métricas en un solo número es difícil de interpretar. Por ejemplo, un "índice de salud" que promedia con distintos pesos la disponibilidad, la latencia y el uso de CPU: si la disponibilidad empeora y la latencia mejora, el índice puede no moverse, y aunque se mueva, nadie sabe qué cambió. Es preferible un SLI por propiedad, calculado de forma directa.

Evitar absolutos. "Siempre disponible" o "escala infinitamente sin aumentar la latencia" no son requisitos realistas, y acercarse a ellos es caro e innecesario.

Tener la menor cantidad posible de SLOs. Solo los suficientes para cubrir los atributos del sistema. Si un SLO nunca sirve para ganar una discusión sobre prioridades, probablemente no vale la pena tenerlo.

La perfección puede esperar. Es mejor empezar con un objetivo laxo y ajustarlo que con uno estricto que haya que relajar al descubrir que es inalcanzable.

2.3.3 Margen de seguridad y no sobrecumplir

Dos tácticas ayudan a fijar expectativas realistas.[19] La primera es usar internamente un SLO más estricto que el que se publica a los usuarios: la diferencia da margen para corregir problemas crónicos antes de que se vuelvan visibles afuera. La segunda es no sobrecumplir, porque los usuarios construyen sobre lo que el servicio realmente ofrece, no sobre lo que promete. Si un servicio rinde mucho mejor que su SLO, los usuarios empiezan a depender de ese rendimiento.

El caso de Chubby. Chubby es el servicio de *locks* distribuido de Google. Usa Paxos (clase 8) para mantener los *locks* de forma consistente entre centros de datos, y otros servicios lo usan para elegir líder (de cinco réplicas de un trabajo, decide cuál hace el trabajo real) y para guardar datos que deben ser consistentes, como la tabla que traduce nombres de servicios a direcciones IP y puertos.[20] Su instancia global tiene una réplica en cada región geográfica, y las caídas reales de esa instancia eran tan poco frecuentes que los equipos empezaron a agregar dependencias suponiendo que nunca se caería.

[17] Beyer et al. (2016), cap. 4.
[18] Beyer et al. (2016), cap. 4, sección *Choosing Targets*.
[19] Beyer et al. (2016), cap. 4.
[20] Beyer et al. (2016), cap. 2.

Esa confiabilidad daba una falsa sensación de seguridad: cuando Chubby global finalmente fallaba, los servicios que dependían de él no sabían funcionar sin él, y sus fallas terminaban siendo visibles para los usuarios finales.[21]

La solución fue asegurar que Chubby global cumpla su SLO, pero sin superarlo demasiado. Si en un trimestre ninguna falla real bajó la disponibilidad hasta el objetivo, se provoca una caída controlada, apagando el sistema a propósito. Así, las dependencias irrazonables aparecen poco después de agregarse, cuando todavía son fáciles de corregir, y no años más tarde en medio de una caída real; los dueños de los servicios dependientes se ven obligados a enfrentar temprano la realidad de los sistemas distribuidos: todo componente remoto puede fallar. Hay otras formas de no sobrecumplir: rechazar a propósito una parte de las solicitudes, o diseñar el sistema para que no sea más rápido cuando tiene poca carga. La caída provocada de Chubby se parece a un experimento bajo falla (sección 5), aunque su propósito es otro: no busca validar cómo se comporta el sistema, sino fijar expectativas en quienes dependen de él.[22]

2.3.4 La ventana

Todo SLO se evalúa sobre una ventana de tiempo, y la ventana cambia el significado del incumplimiento: una mala hora no equivale a una degradación sostenida durante semanas. Hay dos opciones:[23]

- **Ventana fija**, alineada con el calendario (del 1 al 30 del mes). Es simple, pero se reinicia de golpe: una caída el día 31 y otra el día 2 caen en ventanas distintas, aunque el usuario las recuerde juntas.
- **Ventana deslizante**, que mira siempre los últimos N días. Se parece más a la memoria del usuario: el presupuesto se recupera poco a poco a medida que los errores viejos salen de la ventana.

Una ventana deslizante de cuatro semanas, o de 30 días, es un buen intervalo de uso general, complementada con resúmenes semanales para priorizar tareas y reportes trimestrales para planificar proyectos.[24] Ventanas de 7 o 14 días no coinciden con la memoria del cliente ni con los ciclos de planificación. Una de 90 días es demasiado larga: se podría gastar el 90 % del presupuesto en un solo día y cumplir igual el SLO.

2.4 SLA: el acuerdo

El SLA agrega al SLO una consecuencia: un reintegro, una penalidad o una cláusula de rescisión. La forma más simple de distinguirlos es preguntar qué pasa si no se cumple: si no hay una consecuencia explícita, se trata casi con seguridad de un SLO.[25]

Un SLA es una decisión de negocio y legal: los equipos comerciales y legales eligen las consecuencias y las penalidades. Al equipo técnico le corresponden dos tareas. La primera es ayudar a entender cuán probable y cuán difícil es cumplir los SLOs que el SLA contiene. La segunda es que los SLIs del acuerdo puedan medirse de forma objetiva, porque de lo contrario cada incumplimiento termina en una discusión. Conviene ser conservador en lo que se promete: cuantos más clientes tiene un SLA, más difícil es cambiarlo si resulta imprudente.[26]

[21] Beyer et al. (2016), cap. 4, recuadro *The Global Chubby Planned Outage*.
[22] Beyer et al. (2016), cap. 4, nota 3.
[23] Majors et al. (2022), cap. 13.
[24] Beyer et al. (2018), cap. 2; Majors et al. (2022), cap. 13.
[25] Beyer et al. (2016), cap. 4. En el uso cotidiano, la mayoría de las personas dice "SLA" cuando quiere decir "SLO": quien habla de una "violación del SLA" casi siempre se refiere a un SLO no cumplido. Una violación real de un SLA puede terminar en un juicio por incumplimiento de contrato.
[26] Beyer et al. (2016), cap. 4.

La relación con el SLO interno es la del margen de seguridad de la sección 2.3: el SLA promete menos de lo que el equipo se exige. Para un servicio típico de las herramientas de oficina de Google para empresas (correo, calendario, documentos), un esquema razonable es un objetivo externo trimestral de 99,9 %, respaldado por un objetivo interno más estricto y por un contrato con penalidades si no se cumple el externo.[27] En el ejemplo del generador, el SLA promete 99,5 % por mes mientras el SLO interno exige 99,9 %: el equipo actúa cuando se aparta del 99,9 %, mucho antes de que se active la penalidad.

No todo servicio tiene SLA. La búsqueda de Google no tiene un contrato con el público, y aun así una caída tiene consecuencias (reputación, ingresos por publicidad). Con SLA o sin él, conviene definir SLIs y SLOs para operar el servicio.

2.5 Los objetivos del generador

Con todo lo anterior, una primera versión de los objetivos del generador puede escribirse así:

SLI de disponibilidad. Especificación: proporción de solicitudes válidas de generación de contraseña que reciben una contraseña que cumple la política. Implementación: medido en el balanceador, como

$$\frac{\text{respuestas 200 con una contraseña que cumple la política}}{\text{todas las solicitudes} - \text{respuestas 4xx}} = \frac{\text{2xx buenas}}{\text{2xx} + \text{5xx}}$$

Las 4xx quedan fuera porque son errores del cliente; las 5xx (incluidos los *timeouts* y las conexiones cortadas, que el balanceador registra como 502 o 504) quedan dentro porque son las fallas del servicio. La validación de la política se registra en el servicio, junto con la respuesta.

SLI de latencia. Proporción de solicitudes válidas respondidas en menos de 500 ms, medido en el balanceador.

SLOs. Disponibilidad $\geq$ 99,9 % y latencia $\geq$ 99 %, ambos sobre una ventana deslizante de 30 días.

SLA (ilustrativo). Disponibilidad $\geq$ 99,5 % por mes calendario; por debajo, descuento en la factura.

Los valores son decisiones, no datos: el 500 ms y el 99,9 % tienen que justificarse con lo que el usuario necesita y revisarse con lo que se mida. Quedan registrados como hipótesis hasta tener una línea de base (*baseline*).

3 Error budget

3.1 Definición y cálculo

3.1.1 Definición

Si el SLO no exige el 100 %, existe una cantidad tolerada de falla: el *error budget* (presupuesto de error) es $1 - \text{SLO}$. Insistir en que los SLOs se cumplan el 100 % del tiempo es poco realista e indeseable: reduce la innovación, obliga a soluciones caras y conservadoras, o ambas cosas. Es mejor permitir una tasa de incumplimiento y seguirla de cerca. Dicho de otro modo, el error budget es un objetivo sobre el cumplimiento de los demás objetivos.[28]

[27] Beyer et al. (2016), cap. 3.
[28] Beyer et al. (2016), cap. 4: "An error budget is just an SLO for meeting other SLOs!".

3.1.2 Cuánto es

El presupuesto puede expresarse en tiempo o en eventos. En tiempo, para una ventana de 30 días (43 200 minutos):

SLO	Error budget	Caída total tolerada en 30 días
99 %	1 %	7,2 h
99,5 %	0,5 %	3,6 h
99,9 %	0,1 %	43,2 min
99,95 %	0,05 %	21,6 min
99,99 %	0,01 %	4,32 min
99,999 %	0,001 %	25,9 s

Cada nueve adicional divide el presupuesto por diez.[29] En eventos, el presupuesto es la fracción de solicitudes que puede fallar. Para el generador, con 99,9 % y unos 12,96 millones de solicitudes en 30 días, el presupuesto es de unas 12 960 solicitudes fallidas en 30 días.

La resiliencia de la clase 11, medida contra el presupuesto. El presupuesto permite responder una pregunta que la clase 11 dejó abierta: ¿alcanzan los mecanismos de resiliencia? Se toma el escenario de la clase 11 en el que el servicio de *especiales* falla en el 10 % de las llamadas, y se calcula paso a paso cuánto dura el presupuesto del generador.

1. **Probabilidad de que falle una solicitud.** Cada solicitud llama 5 veces a *especiales*, y solo tiene éxito si las 5 llamadas tienen éxito. Sin reintentos, la probabilidad de que las 5 salgan bien es $0{,}9^5 \approx 0{,}59$, de modo que falla el $41\,\%$ de las solicitudes.
2. **Solicitudes fallidas por hora.** Con 18 000 solicitudes por hora, el 41 % son unas 7 380 solicitudes fallidas por hora.
3. **Duración del presupuesto.** El presupuesto de 30 días es de 12 960 solicitudes fallidas. A 7 380 por hora, se agota en $12\,960/7\,380 \approx 1{,}8$ horas.

Con un reintento por llamada, una llamada falla solo si fallan el intento y el reintento: $0{,}1 \times 0{,}1 = 1\,\%$. La solicitud tiene éxito con probabilidad $0{,}99^5 \approx 0{,}951$, así que falla el 4,9 % de las solicitudes. Repitiendo las cuentas:

Configuración	Solicitudes que fallan	Fallidas por hora	El presupuesto dura
Sin reintentos	41 %	7 380	1,8 h
Con un reintento	4,9 %	882	14,7 h
Lo que admite el SLO	0,1 %	18	30 días

El reintento mejoró la situación unas ocho veces, pero el presupuesto sigue agotándose en menos de un día. El SLO dice algo que la clase 11 no podía decir: la mejora no alcanza. Esa es su utilidad: transforma "está mejor" en "está dentro" o "está fuera".

3.2 Cómo se usa

3.2.1 Alinear incentivos

Entre quienes desarrollan un producto y quienes lo operan hay una tensión estructural: a unos se los evalúa por la velocidad con que entregan funcionalidades, a otros por la confiabilidad. La tensión

[29] El apéndice A de Beyer et al. (2016) tiene una tabla equivalente; por ejemplo, 99,99 % en un año son 52,56 minutos.

aparece en decisiones concretas: cuánto endurecer el software, cuánto probar, cada cuánto desplegar, y cuánto tráfico recibe y durante cuánto tiempo una versión nueva en una prueba *canary* antes de liberarla para todos (la clase 9 presentó esta técnica: exponer primero una fracción pequeña del tráfico real a la versión nueva y comparar errores y latencia antes de ampliar). Sin un criterio objetivo, el equilibrio depende de la habilidad negociadora de cada uno.[30] El error budget es ese criterio, y se usa así:

1. producto define el SLO, es decir, cuánta disponibilidad se espera en el trimestre;
2. un tercero neutral —el sistema de monitoreo— mide la disponibilidad real;
3. la diferencia entre ambos es el margen de falla que todavía se puede gastar en lo que queda del trimestre;
4. mientras quede margen, se pueden liberar versiones nuevas.

El efecto es que el equipo de desarrollo pasa a regularse solo: cuando el presupuesto es amplio, puede tomar más riesgos; cuando está casi agotado, es el propio equipo el que pide más pruebas o despliegues más lentos para no frenar su lanzamiento. Las caídas que no son culpa de nadie —una falla de red, un centro de datos— también consumen presupuesto, y todo el equipo acepta la reducción de despliegues porque la responsabilidad por la disponibilidad es compartida. Y si el equipo tiene problemas para lanzar funcionalidades, puede decidir relajar el SLO para tener más presupuesto.

3.2.2 La política de error budget

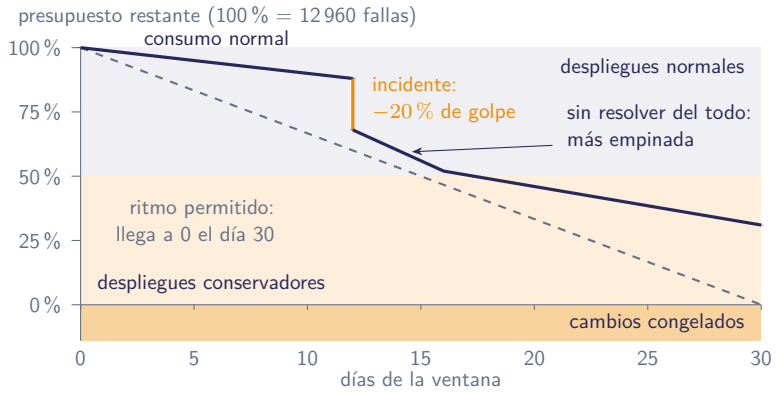
Que el presupuesto se agote tiene que tener consecuencias acordadas *antes*. Una política de error budget especifica qué acciones se toman cuando un servicio consumió todo su presupuesto y quién las toma.[31] Las acciones típicas son: que el equipo de desarrollo priorice los errores de confiabilidad de las últimas cuatro semanas; que se dedique exclusivamente a la confiabilidad hasta volver dentro del SLO; o congelar ciertos cambios en producción hasta recuperar presupuesto. Como todo acuerdo, conviene que esté escrita, que diga quién la aprobó y quién decide si hay desacuerdo.

Además del esquema "todo o nada" (se despliega o no se despliega), hay variantes más graduales, como ir desacelerando las liberaciones a medida que el presupuesto se acerca a cero.[32] Una política posible para el generador (ilustrativa, no tomada de la bibliografía):

- Con más del 50 % de presupuesto: despliegues normales, que siempre incluyen una prueba *canary* y reversión automática si las métricas empeoran.
- Entre 50 % y 0 %: despliegues más conservadores. Se hacen en horarios de poco tráfico (por ejemplo, de madrugada), para que una versión defectuosa afecte a menos usuarios. Y se hacen de forma escalonada: la versión nueva recibe primero un porcentaje pequeño del tráfico (por ejemplo, el 5 %), se observan las métricas durante unos 15 minutos y recién entonces se pasa al escalón siguiente. Es un despliegue mucho más controlado, pero también más lento y tedioso. Además, los errores de confiabilidad tienen prioridad sobre las funcionalidades nuevas.
- Presupuesto agotado: se congelan los cambios que no sean de confiabilidad o de seguridad hasta volver dentro del SLO.
- Un incidente que consume más del 20 % del presupuesto exige un postmortem (sección 7).

[30] Beyer et al. (2016), cap. 3.
[31] Beyer et al. (2018), cap. 2.
[32] Beyer et al. (2016), cap. 3, nota 2.

La figura muestra cómo podría gastarse el presupuesto del generador en una ventana de 30 días (los números son ilustrativos). La diagonal punteada es el ritmo permitido: si el presupuesto se gastara exactamente a ese ritmo, llegaría a cero justo el día 30. Los primeros días el consumo es normal y la línea real baja más despacio que la diagonal. El día 12, un incidente gasta el 20 % de golpe; como el problema no queda resuelto del todo, la línea sigue bajando más empinada hasta el día 16. Poco después cruza el 50 % y la política pasa a pedir despliegues conservadores. La pendiente de la línea en cada momento es el *burn rate* que se usa para alertar (sección 4.3).



Con una ventana deslizante el dibujo es algo distinto: además de gastarse, el presupuesto se recupera a medida que las fallas viejas salen de la ventana.

3.2.3 El costo de cada nueve

El presupuesto también hace visible el costo de los objetivos demasiado altos. Cuando cada solicitud vale lo mismo, el cálculo puede ser directo. Por ejemplo, subir el objetivo de 99,9 % a 99,99 % en un servicio que factura 1 millón de dólares aumenta la disponibilidad en 0,09 %, que vale $1\,\text{M} \times 0{,}0009 = 900$ dólares; si alcanzar el nueve adicional cuesta más que eso, no conviene.[33]

Cuando no hay una traducción directa a dinero, sirve como referencia la falla que el usuario ya sufre por su propia conexión. Aunque el servicio funcione perfecto, una parte de las solicitudes del usuario se pierde en el camino: la red del proveedor de Internet, la señal del celular, el Wi-Fi de su casa. Google midió que, para los proveedores de Internet, esa tasa de error de base está entre 0,01 % y 1 %.[34] Si el servicio, medido desde el usuario, falla menos que eso, el usuario no puede distinguir sus errores de los de su conexión, y bajar todavía más la tasa de error del servicio tiene poco efecto visible.

El SLO no es una aspiración sino una frontera de decisión. Mientras quede error budget, el equipo puede arriesgar; cuando se agota, la confiabilidad pasa al primer lugar. El 100 % casi nunca es el objetivo correcto: es imposible, es caro y el usuario no lo nota.

4 Alertar sobre el SLO: burn rate

4.1 Síntomas, no causas

El monitoreo tradicional alerta sobre lo que es fácil de medir: CPU por encima del 80 %, memoria disponible por debajo del 10 %, disco casi lleno. Esas son causas *posibles* de un problema, y por eso producen muchos falsos positivos: la CPU puede subir porque corre un *backup* o el recolector de basura

[33] Beyer et al. (2016), cap. 3.
[34] Beyer et al. (2016), cap. 3.

(*garbage collector*). Cuando una alarma suena seguido sin que pase nada grave, los equipos aprenden a ignorarla ("esa alerta no importa, el proceso se queda sin memoria cada tanto"), y lo que al principio era una excepción empieza a parecer normal. El resultado es la *fatiga de alertas* (*alert fatigue*): se presta cada vez menos atención a todas, porque la mayoría son falsas alarmas, y el día que una alerta importa, nadie la mira.[35]

Una alerta es útil si cumple dos condiciones: cuando suena, es porque los usuarios realmente están teniendo un problema; y quien la recibe puede hacer algo al respecto.[36] Una alerta por CPU alta no cumple la primera: la CPU del generador puede estar al 90 % mientras todos los usuarios reciben sus contraseñas sin problemas. Una alerta basada en el SLO sí la cumple, y no por casualidad. Vigila el SLI, que se definió justamente como la proporción de solicitudes en las que el usuario recibió lo que esperaba (sección 2.2). Si ese número cae, hay usuarios afectados; si no cae, no los hay, aunque la CPU esté alta.

Lo que la alerta basada en el SLO no dice es *por qué* los usuarios están afectados: dice *qué* pasa (las solicitudes están fallando), no la causa. Para cumplir la segunda condición, quien recibe la alerta necesita poder investigar la causa, y para eso están las trazas, las métricas y los logs de la clase 10.

Las alertas por causa no desaparecen. Nada de esto significa eliminar toda alerta sobre causas. La regla es poner mucho más esfuerzo en detectar síntomas que causas, y alertar por una causa solo cuando es segura e inminente.[37] Un disco casi lleno, un certificado a punto de vencer o un servicio que se quedó sin redundancia (una sola réplica, sin repuesto) todavía no afectan al usuario, pero lo van a afectar con seguridad, y una alerta por síntoma recién sonaría cuando ya es tarde. Esas alertas siguen teniendo sentido, muchas veces como ticket y no como aviso inmediato. Lo que conviene evitar son las alertas por causas que pueden tener o no efecto sobre el usuario, como la CPU alta. Por la misma razón, el monitoreo interno del sistema sigue siendo necesario: sirve para detectar los problemas inminentes y para investigar la causa cuando suena una alerta por síntoma.

4.2 Qué vigila una alerta sobre el SLO

"Alertar sobre el SLO" no significa medir algo nuevo. La alerta usa el mismo SLI que define el SLO, y lo que vigila es el error budget. En el generador, el SLO es "disponibilidad $\geq$ 99,9 % en los últimos 30 días". Una solicitud que no recibe una contraseña válida es una falla, y el presupuesto es el 0,1 % de las solicitudes: unas 12 960 fallas en 30 días. Con esos datos hay tres formas de escribir la alerta:

1. **Alertar cuando el SLO ya no se cumple:** el SLI de los últimos 30 días cae por debajo de 99,9 %. Es la más obvia y la menos útil, porque suena cuando el presupuesto ya está gastado y no queda nada que proteger.
2. **Alertar cuando el saldo del presupuesto baja de un umbral,** por ejemplo, cuando queda menos del 30 %. Avisa antes, pero es tosca. No distingue un problema que gastó el 70 % del presupuesto en tres semanas de uno que lo gastó en una hora y sigue gastando. Además, el umbral se convierte en un nuevo cero: una vez cruzado, el equipo actúa como si el presupuesto estuviera agotado.[38]
3. **Alertar sobre la velocidad de consumo del presupuesto:** mirar solo las fallas recientes (por ejemplo, las de la última hora) y preguntar a qué ritmo se está gastando el presupuesto. Si ese ritmo lo agotaría mucho antes de los 30 días, hay que actuar ahora. Es la forma recomendada, porque la urgencia queda proporcional al daño.

[35] Majors et al. (2022), cap. 12.
[36] Majors et al. (2022), cap. 12.
[37] Beyer et al. (2016), cap. 6, sección *Tying These Principles Together*.
[38] Majors et al. (2022), cap. 13.

El SLO define qué cuenta como falla y cuánto presupuesto hay; la alerta mide a qué velocidad se gasta. Esa velocidad es el *burn rate* de la sección 4.3. Concretamente, para el generador la alerta es una condición como esta:

> **Tasa de error en una ventana** w: solicitudes fallidas en w / solicitudes válidas en w.
>
> **Condición de la alerta inmediata:** tasa de error de la última hora $> 1{,}44\,\%$ **y** tasa de error de los últimos 5 minutos $> 1{,}44\,\%$. (De dónde sale el 1,44 % y para qué sirve la segunda ventana se explica en la sección 4.3.)
>
> **Ejemplo.** En la última hora llegaron 18 000 solicitudes y fallaron 300 (1,67 %); en los últimos 5 minutos llegaron 1 500 y fallaron 30 (2 %). Ambas tasas superan 1,44 %, así que la alerta suena. A ese ritmo, 300 fallas por hora, el presupuesto de 30 días (12 960 fallas) se agotaría en unas 43 horas, mucho antes de los 30 días.

Una alerta *inmediata* (en inglés, *page*) es la que despierta a la persona de guardia (*on-call*): el problema no puede esperar. Una alerta de *ticket* genera una tarea para el día hábil siguiente: el presupuesto se está gastando más rápido de lo debido, pero no tan rápido como para actuar de noche.

4.3 Burn rate

4.3.1 Definición

La tercera opción necesita una medida de velocidad. El *burn rate* (tasa de consumo) mide cuán rápido, en relación con el SLO, el servicio consume el error budget.[39] Con una tasa de error observada e y un SLO de objetivo s:

$$b = \frac{e}{1 - s}$$

Un *burn rate* de 1 consume exactamente el presupuesto en la ventana completa del SLO; uno de b lo agota en T/b, donde T es la ventana.

Para el generador, el SLO es $s = 99{,}9\,\% = 0{,}999$, de modo que $1 - s = 0{,}001$, y la ventana es de 30 días ($T = 720\,\text{h}$). Entonces $b = e/0{,}001$ y el presupuesto se agota en $720\,\text{h}/b$:

Tasa de error (e)	Burn rate	Se agota en	Situación
0,1 %	1	30 días	justo en el límite
0,6 %	6	5 días	degradación sostenida
1,44 %	14,4	50 h	degradación seria
4,9 %	49	14,7 h	*especiales* al 10 %, con un reintento
41 %	410	1,8 h	*especiales* al 10 %, sin reintentos
100 %	1000	43 min	caída total

4.3.2 Elegir la ventana y el umbral

La condición del ejemplo de la sección 4.2 tiene dos números: la ventana (una hora) y el umbral (1,44 %). Elegirlos es un compromiso entre cuatro propiedades que se quieren a la vez:[40]

- **precisión:** que cuando la alerta suena haya un problema real, y no un pico de un minuto sin importancia;
- **cobertura:** que todo problema que pone en riesgo el SLO termine generando una alerta;

[39] Beyer et al. (2018), cap. 5.
[40] Beyer et al. (2018), cap. 5.

- **tiempo de detección:** cuánto tarda en sonar desde que empieza el problema;
- **tiempo de reinicio:** cuánto sigue sonando después de que el problema se resolvió.

Una sola ventana no alcanza. Una ventana corta (5 minutos) detecta rápido, pero es imprecisa: un pico breve dispara la alerta aunque haya gastado una fracción ínfima del presupuesto. Una ventana larga (un día) es precisa, pero detecta tarde y sigue sonando mucho después de resuelto el problema: si hubo 10 minutos de caída total, el promedio del día queda por encima del umbral durante horas, y la guardia no sabe si el problema volvió o es el eco del anterior.

Un solo umbral tampoco. Con un umbral alto, los consumos lentos pasan inadvertidos aunque igual agoten el presupuesto. Si la alerta solo suena cuando el burn rate supera 14,4, un consumo sostenido a burn rate 10 nunca la dispara, y sin embargo agota el presupuesto de 30 días en 3 días.

La solución: varios umbrales, cada uno con dos ventanas. Se definen varias alertas, una por velocidad de consumo: las más rápidas despiertan a la guardia y las más lentas generan un ticket. Y cada alerta exige dos condiciones a la vez: que el umbral se supere en una ventana *larga*, lo que confirma que el consumo es significativo, y en una ventana *corta*, lo que confirma que sigue ocurriendo ahora. La ventana corta hace que la alerta deje de sonar pocos minutos después de resuelto el problema.

Los parámetros recomendados para un SLO de 99,9 % son:[41]

Severidad	Ventana larga	Ventana corta	Burn rate	Presupuesto consumido
Aviso (*page*)	1 h	5 min	14,4	2 %
Aviso (*page*)	6 h	30 min	6	5 %
Ticket	3 días	6 h	1	10 %

La última columna indica cuánto presupuesto se consumió cuando la alerta suena. Si un burn rate b se mantiene durante un tiempo t, la fracción del presupuesto consumida es:

$$\text{presupuesto consumido} = \frac{b \cdot t}{T}$$

Para la primera alerta, t es la ventana larga (1 hora): $14{,}4 \times 1\,\text{h}/720\,\text{h} = 2\,\%$. Para la segunda, $6 \times 6\,\text{h}/720\,\text{h} = 5\,\%$; para la de ticket, $1 \times 72\,\text{h}/720\,\text{h} = 10\,\%$. Como regla práctica, la ventana corta mide la duodécima parte de la larga. La lectura es operativa: si en la última hora se consumió el 2 % del presupuesto mensual (y sigue ocurriendo en los últimos 5 minutos), alguien tiene que mirar ahora; si en tres días se consumió el 10 %, alcanza con un ticket para el día hábil siguiente.

Tiempo de detección. Si la tasa de error salta de golpe a e, la alerta de la ventana de una hora se dispara cuando el promedio de la hora alcanza $14{,}4 \times 0{,}1\,\% = 1{,}44\,\%$, es decir, después de $1{,}44\,\%/e$ de hora. Con el servicio de *especiales* fallando el 10 % de las llamadas y sin reintentos ($e = 41\,\%$), la alerta llega en unos 2 minutos; ante una caída total, en menos de un minuto. Una degradación leve, en cambio, tarda más en alertar, y es lo correcto: consume el presupuesto más despacio.

4.3.3 Servicios con poco tráfico

Con poco tráfico, una sola solicitud fallida parece una catástrofe. Por ejemplo, un servicio con 10 solicitudes por hora y un SLO de 99,9 % solo puede tener unas siete solicitudes fallidas en 30 días. Un único fallo en una hora es una tasa de error horaria del 10 %, un burn rate de 100, y consume el 13,9 %

[41] Beyer et al. (2018), cap. 5, tabla 5-8.

del presupuesto mensual.[42] Hay varias opciones: generar tráfico artificial para tener más señal; medir juntos varios servicios pequeños que suelen fallar juntos (por ejemplo, porque comparten una base de datos); cambiar el producto para que un fallo individual tenga menos impacto (reintentos del cliente con *backoff* y *jitter*, caminos alternativos); o bajar el SLO o agrandar la ventana, que es una decisión de producto. En la práctica, se combinan.

4.3.4 Contar minutos o contar solicitudes

Un SLI se puede calcular contando solicitudes, como el de la sección 2.2, o contando minutos: cada minuto se marca como bueno o malo según cómo les fue a sus solicitudes, y se cuentan los minutos malos. La segunda forma es la que permite informar que un servicio "estuvo caído 12 minutos". La diferencia parece de forma, pero cambia cuánto presupuesto se descuenta y, por lo tanto, cuándo suenan las alertas.[43] Con el SLO de disponibilidad del generador (99,9 % en 30 días), las dos formas quedan así:

- **Por tiempo:** cada minuto se marca como bueno o malo. Un minuto es malo si en él falló el 0,1 % o más de las solicitudes. El presupuesto es de 43,2 minutos malos en 30 días.
- **Por eventos:** cada solicitud es buena o mala. El presupuesto es de 12 960 solicitudes fallidas en 30 días.

Un problema chico que parece grande. Durante una hora falla el 1 % de las solicitudes: 180 de las 18 000 que llegan en esa hora.

- Contando solicitudes, se consumen $180/12\,960 \approx 1{,}4\,\%$ del presupuesto. Es un problema menor, que se puede atender con calma.
- Contando minutos, cada uno de los 60 minutos tuvo más del 0,1 % de fallas, así que los 60 son malos. Pero el presupuesto es de 43,2 minutos: esa hora lo agotó y el SLO quedó incumplido. El método trata cada minuto malo como si hubieran fallado todas sus solicitudes, cuando falló una de cada cien.

Un problema real que no se ve. Ahora el SLO de latencia del generador: el 99 % de las solicitudes debe responderse en menos de 500 ms, es decir, el presupuesto es el 1 % de solicitudes lentas. Contando por tiempo, un minuto es bueno si su percentil 95 (p95) es menor que 500 ms. Durante 30 días, de forma constante, el 4 % de las solicitudes tarda más de 500 ms.

- Contando minutos, en cada minuto solo el 4 % de las solicitudes es lenta, así que el p95 sigue por debajo de 500 ms: todos los minutos son buenos y el presupuesto queda intacto. El método no ve el problema.
- Contando solicitudes, el 4 % de solicitudes lentas es cuatro veces el 1 % permitido: el presupuesto se agota en una cuarta parte de la ventana, unos 7,5 días, y el SLO queda incumplido.

Contar minutos exagera algunos problemas y oculta otros, porque reduce cada minuto a un sí o un no. Contar solicitudes descuenta exactamente lo que vivieron los usuarios. Esto importa especialmente en un sistema distribuido, donde lo habitual no es la caída total sino la degradación parcial (en inglés, *brownout*): una dependencia lenta o con errores afecta a una fracción de las solicitudes, como en los escenarios de la clase 11.

Cada forma tiene su uso:

[42] Beyer et al. (2018), cap. 5. El texto original dice "1,000x burn rate", pero con sus propios datos la cuenta da 100.
[43] Majors et al. (2022), cap. 13.

- **Contar solicitudes** conviene para los SLOs con los que el equipo opera el servicio y para calcular el burn rate, sobre todo con objetivos exigentes (99,9 % o más).
- **Contar minutos** conviene cuando el resultado tiene que ser fácil de explicar y de auditar, como en un SLA o en una página de estado ("el servicio estuvo caído 12 minutos este mes"). Funciona mejor con mucho tráfico por minuto y con un umbral pensado para detectar caídas, no degradaciones leves.

4.3.5 Un caso: la alerta que nadie creía

En Honeycomb, una empresa de herramientas de observabilidad, el servicio que recibe e ingiere los datos de todos sus clientes tenía un SLO con un objetivo de 99,995 %, y sobre él una alerta de consumo del presupuesto como la de la sección 4.2.[44] El equipo todavía no confiaba en esas alertas: las mandaba a un canal de prueba y seguía guiándose por las alertas tradicionales.

Una madrugada, el servicio tuvo una degradación parcial: durante 20 minutos falló alrededor del 1,5 % de los datos entrantes. Con un presupuesto de apenas 0,005 %, eso es un consumo 300 veces más rápido que el permitido, y la alerta del SLO sonó a la 1:29. La persona de guardia vio que el servicio parecía recuperado y lo tomó como un pico pasajero. La degradación se repitió, y recién cuando la alerta sonó por cuarta vez, a las 9:55, se declaró un incidente. El monitoreo tradicional, que necesitaba varias sondas fallidas seguidas, nunca detectó nada.

La causa era una pérdida de memoria introducida en un despliegue: las máquinas del clúster se quedaban sin memoria, caían y se reiniciaban casi al mismo tiempo. Ninguna alerta por memoria la habría detectado, porque en ese sistema quedarse sin memoria de vez en cuando era normal; lo anómalo era el efecto sobre los usuarios, y eso es lo que mide el SLO. Al cerrarse el incidente, el cumplimiento había caído a 99,97 % contra un objetivo de 99,995 %: se había gastado más de cinco veces el presupuesto del período. Después de este caso, el equipo pasó a usar las alertas de consumo del presupuesto como su alerta principal y eliminó buena parte de las tradicionales. Es la experiencia de una empresa de herramientas de observabilidad, que defiende esa práctica; el SRE Book es más prudente y conserva las alertas por causas seguras e inminentes (sección 4.1).

4.4 Qué hacer cuando suena

Una alerta de burn rate inicia una investigación. Antes de actuar conviene distinguir qué patrón de consumo se está viendo: un consumo gradual y esperable, consumos en ráfagas, o un incidente que gasta de golpe una parte grande del presupuesto. Comparar con el historial ayuda a decidir la urgencia, y el patrón da pistas sobre la causa: gastar el presupuesto de una vez sugiere un tipo de falla distinto de gastarlo de a poco.[45] Si la alerta corresponde a un incidente, empieza la sección 6.

> Se alerta sobre lo que vive el usuario, no sobre lo que podría causarlo. El burn rate convierte el SLO en una alarma con dos propiedades deseables: urgencia proporcional a la gravedad (una caída total avisa en segundos, una degradación leve deja un ticket) y silencio cuando el problema se resuelve.

5 Validar: experimentos bajo falla

[44] Majors et al. (2022), cap. 12, sección *Changing Culture Toward SLO-Based Alerts: A Case Study*, págs. 135–137.
[45] Majors et al. (2022), cap. 13.

5.1 Por qué no alcanza con probar los componentes

La clase 11 configuró *timeouts*, reintentos y *circuit breakers*. ¿Cómo se sabe que funcionan? Las pruebas clásicas —unitarias, de integración, de sistema en escenarios nominales— son necesarias, pero suelen suponer tiempos ideales, usan dobles de prueba (*mocks*) demasiado limpios y rara vez modelan pérdidas, reordenamientos o particiones. Prueban funciones, no propiedades del sistema completo.

Hay problemas que solo aparecen cuando todas las piezas funcionan juntas: los reintentos que se multiplican hasta saturar una dependencia, las fallas en cascada, la lentitud de un servicio que bloquea los hilos de otro (todos vistos en la clase 11). La razón de fondo es que la seguridad no se compone: dos servicios pueden comportarse bien por separado sin que su combinación lo haga.[46] Por ejemplo, un cliente con un *timeout* de 50 ms llama a dos proveedores que responden en 20 ms en promedio y casi nunca superan los 30 ms. Llamar a cualquiera de los dos es seguro. Llamar a ambos, uno después del otro, cumple el plazo en promedio (40 ms), pero cada vez que los dos responden algo más lentos de lo habitual a la vez, la suma pasa los 50 ms y el cliente falla. El generador es una versión ampliada del mismo problema: veinte llamadas por solicitud, cada una razonable por separado.

Tampoco alcanza con un entorno de pruebas (*staging*). Nunca es una copia a escala de producción: tiene menos servidores, menos tráfico y una red sin congestión, y muchos problemas solo aparecen con la carga, la cantidad de instancias y la red de producción.[47]

5.2 Qué es Chaos Engineering

5.2.1 Definición

Chaos Engineering es la disciplina de hacer experimentos sobre un sistema para ganar confianza en que puede soportar condiciones turbulentas en producción.[48] Es una disciplina empírica: no intenta deducir de un modelo qué *debería* hacer el sistema ante una falla, sino provocar la falla de forma controlada y observar qué *hace*. El nombre engaña: no se trata de romper cosas al azar, sino de experimentar con método.

5.2.2 Por qué provocar fallas a propósito

Provocar fallas en un sistema que funciona parece una mala idea. Hay tres razones para hacerlo.[49]

Los sistemas pierden margen sin que nadie lo decida. Cada optimización, tomada por separado, es razonable: dimensionar el pool de hilos para la carga habitual, quitar una réplica que casi nunca se usa, ajustar un *timeout* para responder más rápido. Pero cada una consume parte del margen que el sistema tenía para absorber imprevistos, y nadie lleva la cuenta del total. Un sistema muy eficiente funciona perfecto en condiciones normales y, cuando algo falla, no tiene de dónde sacar holgura: tiende a romperse todo junto. En el generador, un pool de hilos justo para 5 solicitudes por segundo anda bien hasta el día en que *especiales* se vuelve lento (clase 11). Un experimento mide cuánto margen queda antes de que lo mida un incidente.

Lo que nunca falla no se sabe manejar. Los equipos aprenden a resolver los problemas que ven seguido. Si un componente casi nunca falla, nadie sabe cómo se comporta el resto del sistema sin él, ni cómo recuperarlo, y cuando por fin falla, la situación es peor que si fallara a menudo. Es lo que pasó

[46] Nygard (2018), cap. 17.
[47] Nygard (2018), cap. 17.
[48] *Principles of Chaos Engineering* (2019).
[49] Nygard (2018), cap. 17.

con Chubby (sección 2.3): su alta confiabilidad hizo que otros servicios dependieran de él sin estar preparados para su ausencia. Paradójicamente, cuanto mejor funciona un sistema, menos información da sobre qué pasaría si dejara de funcionar.

La falla frecuente y pequeña se vuelve rutina. Por eso conviene introducir fallas pequeñas y controladas con regularidad, del mismo modo que un edificio hace simulacros de evacuación: no para que haya un incendio, sino para que, si lo hay, todos sepan qué hacer y las salidas estén despejadas. La recuperación ante una falla que se ensaya todas las semanas deja de ser un evento extraordinario.

El ejemplo más conocido. Netflix desarrolló *Chaos Monkey*, una herramienta que cada tanto elige al azar un grupo de servidores de un mismo servicio y apaga uno. El servicio debería seguir funcionando con los demás; si no lo hace, el equipo dueño tiene que corregirlo. La idea surgió cuando Netflix migró a la nube y a muchos servicios pequeños: en la nube, los servidores se caen con la frecuencia suficiente para ser un problema, pero no tanto como para que cada servicio quede probado. Apagarlos a propósito hizo que esa falla se volviera rutina. Después vinieron otras herramientas similares, por ejemplo, para agregar latencia a las llamadas entre servicios.[50]

5.2.3 Los principios

Un experimento tiene cuatro pasos:[51]

1. definir el *steady state* (estado estable): una salida medible del sistema que indica comportamiento normal;
2. formular la hipótesis de que ese estado estable se mantiene tanto en un grupo de control como en un grupo experimental;
3. introducir variables que reflejen eventos del mundo real: servidores que caen, discos que fallan, conexiones que se cortan;
4. intentar refutar la hipótesis buscando una diferencia en el estado estable entre ambos grupos.

A esos pasos se suman algunas recomendaciones: formular la hipótesis sobre lo que se ve desde afuera del sistema, no sobre su funcionamiento interno; elegir las fallas a inyectar entre las que ocurren en la realidad, priorizando las más frecuentes o las de mayor impacto; experimentar, cuando sea seguro, en producción, porque es el único entorno realista; automatizar los experimentos para que corran de forma periódica; y limitar el daño posible, el radio de impacto o *blast radius*.

El steady state es el SLI. El primer paso es exactamente lo que la sección 2 construyó: una salida medible, vista desde afuera, que define lo normal. Un equipo que ya tiene SLIs tiene su *steady state*; uno que no los tiene no sabe qué observar y solo genera ruido. Por eso el orden de esta clase: primero medir, después experimentar.

5.3 Diseñar y ejecutar un experimento

5.3.1 La hipótesis

Conviene escribir la hipótesis como algo que el sistema debe seguir cumpliendo aunque falle una parte, y que se pueda observar desde afuera.[52] La hipótesis detrás de *Chaos Monkey* era "un servicio con varios

[50] Nygard (2018), cap. 17.
[51] *Principles of Chaos Engineering* (2019).
[52] Nygard (2018), cap. 17.

servidores no se ve afectado si uno se cae" (los primeros experimentos la refutaron). Una hipótesis vaga como "queremos ver si aguanta" no sirve: no dice qué observar ni qué resultado la refuta.

Con la hipótesis escrita, hay dos verificaciones previas:

- **¿Se puede medir hoy el steady state?** Puede hacer falta ajustar la medición antes de empezar. Si el SLI del generador se midiera solo con los logs del propio servicio, un experimento que hace caer el proceso no mostraría ninguna falla: las solicitudes que no llegan al proceso no quedan en sus logs.
- **¿Qué resultado refutaría la hipótesis?** No toda falla alcanza: si las solicitudes vienen de Internet, siempre hay algunas que fallan por la red del usuario. Por eso se compara contra un grupo de control que no recibe la perturbación: lo que importa es la *diferencia* entre ambos grupos.

Al final de esta sección hay un *template* para documentar el experimento completo, a partir de la hipótesis (sección 5.3.5).

5.3.2 Qué inyectar

El tercer paso de un experimento (sección 5.2.3) es introducir variables que reflejen eventos del mundo real. En la práctica, esas variables son fallas que se provocan a propósito, lo que se llama inyectar fallas (*fault injection*). Hay tres inyecciones básicas:[53]

Apagar servidores. La más simple: comprueba que el servicio sigue funcionando con los servidores que quedan.

Agregar latencia. Encuentra debilidades que apagar un servidor no muestra: servicios que, ante un *timeout*, devuelven un error cuando podrían tener una alternativa útil, y errores que solo aparecen cuando las respuestas llegan en un orden distinto del habitual.

Hacer fallar una llamada entre servicios. Se marca una solicitud al entrar al sistema con una indicación del tipo "cuando el generador llame a *especiales*, esta llamada debe fallar". La marca viaja con la solicitud, igual que el contexto de una traza en la clase 10, y en el punto indicado la llamada se reporta como fallida sin hacerse. Así se puede hacer fallar una sola llamada, para una fracción pequeña de solicitudes.

A esas se suman las perturbaciones de red —pérdida de paquetes, particiones, ancho de banda reducido—, que suelen inyectarse con un proxy entre los servicios, y las de recursos (CPU, disco). La distinción más importante es la de la clase 11: una caída es una falla *rápida*; la latencia, una falla *lenta*, y la lenta es la más peligrosa. Muchos sistemas toleran bien que una dependencia desaparezca y muy mal que se vuelva lenta, por lo que conviene probar ambas.

Dónde inyectar. Al principio, elegir al azar funciona: casi cualquier falla inyectada encuentra algo. Cuando lo fácil ya está corregido, la cantidad de combinaciones posibles (qué llamada, qué servidor, qué tipo de falla) es demasiado grande para probarlas todas, y hay que elegir. La forma de hacerlo es mirar el árbol de llamadas de una solicitud exitosa —por ejemplo, en una traza— y preguntar qué pasa si se quita cada una de esas llamadas. También conviene estudiar las veces en que una falta *no* se convirtió en falla: algo en el sistema la contuvo, y conviene saber qué.[54]

[53] Nygard (2018), cap. 17.
[54] Nygard (2018), cap. 17.

5.3.3 Seguridad del experimento

Antes de experimentar en producción hay prerrequisitos.[55] El primero es que el experimento no puede causar un daño que el negocio no tolere: si cada solicitud fallida es irrecuperable (un pago perdido, un dato médico), experimentar en producción no es una opción. Los demás:

- **Acotar el *blast radius***, es decir, la cantidad de clientes afectados y cuánto se los afecta. Se puede empezar con algo tan simple como "falla una de cada 10 000 solicitudes" y pasar después a criterios más finos.
- **Poder seguir una solicitud a través de los servicios** y saber si al final tuvo éxito. Si tuvo éxito, la traza muestra qué redundancia lo salvó; si falló, dónde.
- **Saber cómo se ve un sistema sano** (el SLI y su línea de base), con suficiente detalle para notar un cambio pequeño.
- **Tener un plan de recuperación.** El sistema no necesariamente vuelve solo a un estado sano cuando se apaga la perturbación.

A esto conviene sumar una escalera de entornos (una prueba local y determinista, luego un entorno de integración, luego un porcentaje pequeño de tráfico de producción y solo al final producción con un *blast radius* mayor), comunicar el experimento a los equipos involucrados y fijar el criterio de aborto antes de empezar. Con SLOs, el criterio de aborto se escribe en el mismo idioma que las alertas: el experimento se detiene si el burn rate de la ventana corta supera el que se esperaba.

El experimento consume presupuesto. Un experimento en producción gasta error budget, y ese costo se calcula de antemano. Si se perturba el 1 % del tráfico y todas esas solicitudes fallan, la tasa de error es del 1 %, el burn rate es 10 y treinta minutos de experimento consumen $10 \times 0{,}5\,\text{h}/720\,\text{h} \approx 0{,}7\,\%$ del presupuesto mensual. Si queda poco presupuesto, la política de la sección 3 puede postergar el experimento: el error budget también decide cuándo se experimenta.

Dos advertencias. La primera es una prueba de Google que se salió de control.[56] Para descubrir dependencias ocultas sobre una base de datos de prueba, se planeó bloquear el acceso a una sola de un centenar de bases. A los pocos minutos, numerosos servicios dependientes informaron que usuarios internos y externos no podían acceder a sistemas clave. El equipo abortó de inmediato, pero el procedimiento para revertir los permisos falló, y restablecer el acceso llevó cerca de una hora. Las lecciones: la prueba se había revisado y se creía bien acotada, pero se entendía mal la interacción entre los sistemas dependientes; no se siguió el proceso de respuesta a incidentes, que tenía pocas semanas y no se había difundido; y el procedimiento de reversión nunca se había probado. Desde entonces se exige probar la reversión antes de pruebas de esa escala.

La segunda es más modesta. Una ingeniera recién llegada a una empresa avisó por correo que empezaría a experimentar en el entorno de pruebas y que quien no quisiera participar podía pedir quedar afuera. Casi nadie respondió; tomó el silencio como consentimiento, y el entorno de pruebas quedó inutilizable una semana. La lección: el silencio no es consentimiento, y los primeros experimentos tienen que ser pequeños y acordados con los equipos afectados.[57]

[55] Nygard (2018), cap. 17.
[56] Beyer et al. (2016), cap. 13, *Test-Induced Emergency*.
[57] Nygard (2018), cap. 17, recuadro *Introducing Chaos to Your Neighbors*.

5.3.4 Automatizar y repetir

En el mejor caso, un experimento es aburrido: se inyecta la falla y el sistema sigue funcionando como siempre.[58] Cuando un experimento encuentra una debilidad, hay que corregir ese caso y buscar qué otras partes del sistema tienen el mismo tipo de problema. Después, la prueba se automatiza para que corra de forma periódica, con límites: un experimento que apaga servidores no debería apagar el último que queda, y uno que hace fallar la llamada a *especiales* no debería hacer fallar a la vez la alternativa que se usa cuando *especiales* no responde.

Las personas también fallan. El caos no se limita al software. Las organizaciones de alta confiabilidad hacen simulacros para encontrar debilidades del lado humano: se declara "incapacitada" a una parte del equipo durante un día y se observa si el trabajo puede seguir. Las primeras veces aparecen procesos que dependen de una sola persona o de información que solo alguien conoce, y se registran y corrigen como en un postmortem. También acá hace falta una forma de abortar: una palabra clave que indique "esto no es parte del simulacro".[59] Google hace periódicamente ejercicios de este tipo, que combinan fallas técnicas y de personas, con el nombre DiRT (*Disaster Recovery Testing*, pruebas de recuperación ante desastres), e incluye en ellos la práctica de la gestión de incidentes.[60]

5.3.5 Documentar el experimento

Antes de ejecutar un experimento conviene escribir su diseño completo. Escribirlo obliga a decidir de antemano lo que en medio del experimento se improvisaría mal —cuánto tráfico se afecta, qué señal lo detiene y cómo se vuelve atrás—, permite compartirlo con los equipos afectados antes de empezar y, al terminar, se convierte en el informe del resultado. El siguiente *template* reúne los elementos vistos en esta sección; la sección 5.4 lo aplica al generador.

Campo	Contenido
Pregunta	Qué mecanismo o supuesto se quiere validar.
Steady state	SLI usado y su valor en la línea de base.
Hipótesis	Qué se espera que ocurra con el *steady state*, escrito de forma refutable.
Perturbación	Qué se inyecta, dónde, con qué magnitud y por cuánto tiempo.
Grupos	Qué tráfico es control y qué tráfico es experimental.
Blast radius	Qué porcentaje del tráfico y qué componentes pueden verse afectados.
Costo esperado	Cuánto error budget se espera consumir.
Criterio de aborto	Qué señal detiene el experimento, y quién lo detiene.
Recuperación	Qué hay que reiniciar, revertir o limpiar al terminar.
Resultado y conclusión	Qué se observó, si la hipótesis se sostuvo y qué acciones se derivan.

5.4 Un experimento sobre el generador

Pregunta. ¿Los *timeouts* y el *breaker* configurados en la clase 11 contienen una dependencia lenta?

Steady state. SLI de disponibilidad y SLI de latencia del generador, medidos en el balanceador;

[58] Nygard (2018), cap. 17.
[59] Nygard (2018), cap. 17, sección *Disaster Simulations*.
[60] Beyer et al. (2016), caps. 13 y 14.

ambos estables en la línea de base.

Hipótesis. Si las llamadas al servicio de *especiales* del 1 % de las solicitudes sufren 2 s de latencia adicional, esas solicitudes fallan rápido, con un error explícito, y las solicitudes no afectadas mantienen sus dos SLIs dentro de la línea de base.

Perturbación y grupos. Latencia inyectada solo en las llamadas de las solicitudes marcadas (grupo experimental, 1 %); el resto es el grupo de control. Duración: 30 minutos.

Costo esperado. Burn rate cercano a 10; alrededor del 0,7 % del presupuesto mensual.

Aborto. Burn rate de la ventana de 5 minutos mayor que 14,4 (señal de que el daño excede al 1 % marcado), o latencia del grupo de control fuera de la línea de base.

Qué refutaría la hipótesis. Que la latencia del grupo de control suba: los hilos de las solicitudes lentas se comparten con los demás (falta un *bulkhead*, o el *timeout* es demasiado largo). Que las solicitudes marcadas tarden 2 s en fallar: el *timeout* no está activo.

Un segundo experimento muestra el valor de estudiar lo que no se esperaba. Si se repite el mismo diseño sobre el servicio de *dígitos* con la hipótesis "solo se ven afectadas las llamadas a *dígitos*", la hipótesis se refuta: las llamadas a *minúsculas* también se vuelven lentas o fallan, porque *minúsculas* consulta a *dígitos* por dentro. El experimento descubre la dependencia que no figuraba en el diagrama, que es exactamente lo que buscaba la prueba de Google descrita antes, y deja una acción concreta: documentar la dependencia y revisar que el *timeout* de *minúsculas* contemple el de *dígitos* (la cadena de plazos de la clase 11).

La resiliencia no se supone: se valida con experimentos. Un experimento válido tiene un *steady state* medible (el SLI), una hipótesis que puede refutarse, una perturbación acotada, un costo calculado en error budget y un criterio para abortar. "Apagar cosas y mirar" no produce evidencia.

6 Responder: gestión de incidentes

6.1 Qué es un incidente y cuándo declararlo

Un incidente es un evento que degrada o amenaza el servicio —su disponibilidad, su latencia, la corrección de sus respuestas, la seguridad de sus datos— y que requiere una respuesta coordinada. No todo error lo es: una solicitud fallida aislada consume presupuesto, pero no exige movilizar a nadie. Con SLOs, la frontera se vuelve más clara: una alerta de burn rate de severidad *page* es, en general, el comienzo de un incidente.

Conviene fijar de antemano condiciones claras para declararlo. Por ejemplo, las de un equipo de Google: si hace falta involucrar a un segundo equipo para resolver el problema, si la caída es visible para los clientes o si el problema sigue sin resolverse después de una hora de análisis concentrado, es un incidente.[61] Y es mejor declararlo temprano y cerrarlo al encontrar una solución simple que poner en marcha el proceso horas después, cuando el problema ya creció.

6.2 El incidente no gestionado

Un relato muestra qué pasa sin un proceso. La persona de guardia recibe una alerta: uno de los cinco centros de datos dejó de atender tráfico. Mientras investiga, cae un segundo y un tercero, y los dos restantes se sobrecargan. Revierte a la versión anterior sin éxito, llama a quien escribió el código (son las 3:30 de la madrugada en su zona horaria), otros compañeros empiezan a "mirar" desde sus

[61] Beyer et al. (2016), cap. 14, *When to Declare an Incident*.

terminales, los directivos llaman pidiendo explicaciones y fechas. Sin que nadie lo coordine, un ingeniero convocado por otro despliega en producción una optimización que se le ocurrió en el momento; los servidores se reinician y mueren.[62]

Todos estaban haciendo su trabajo como lo entendían. Hay tres peligros:

Foco exclusivo en el problema técnico. Quien sabe arreglar el sistema está ocupado arreglándolo y no puede pensar en el cuadro general ni en cómo mitigar.

Mala comunicación. Por lo mismo, nadie sabe qué hacen los demás; los responsables del negocio se enojan y quienes podrían ayudar no se aprovechan.

Trabajo por cuenta propia. Alguien hace cambios con la mejor intención, sin coordinar con quien está a cargo, y empeora la situación.

6.3 Cómo se gestiona un incidente

6.3.1 Roles

La forma habitual de gestionar incidentes en la industria del software está tomada de los servicios de emergencia (bomberos, defensa civil), que necesitan coordinar a muchas personas que no trabajan juntas a diario.[63] Su primer elemento es una separación explícita de responsabilidades. Cada persona sabe qué rol tiene y no invade el de otro; de forma algo contraintuitiva, eso da *más* autonomía, porque nadie necesita cuestionar a sus compañeros.

Rol	Responsabilidad
Incident commander	Mantiene el estado general del incidente, organiza al equipo y asigna responsabilidades; tiene todos los roles que no delegó. Remueve obstáculos.
Operaciones	Aplica las herramientas operativas para responder. Es el *único* grupo que modifica el sistema durante el incidente.
Comunicación	Es la cara pública del equipo: emite actualizaciones periódicas para el equipo y los interesados y suele mantener el documento del incidente.
Planificación	Se ocupa de lo de más largo plazo: registrar errores, organizar los relevos, anotar en qué se apartó el sistema de lo normal para revertirlo al final.

Si la carga de alguien se vuelve excesiva, pide más personas y delega, incluso creando subincidentes. En un equipo pequeño una persona puede tener varios roles; lo importante es que esté explícito quién tiene cada uno.

6.3.2 Puesto de mando, documento vivo y traspaso

Los demás elementos del proceso son:[64]

- **Un puesto de mando reconocido**: un lugar, físico o un canal de chat, donde los interesados saben que pueden encontrar al *incident commander*. Los registros del canal sirven después para el postmortem.

[62] Beyer et al. (2016), cap. 14.
[63] Beyer et al. (2016), cap. 14. El modelo de origen es el *Incident Command System* de la gestión de emergencias de los Estados Unidos.
[64] Beyer et al. (2016), cap. 14 y apéndice C.

- **Un documento vivo del incidente**, cuya responsabilidad principal es del *incident commander*. Puede estar desordenado, pero tiene que ser funcional, y conviene usar un *template* con lo más importante arriba. Un buen documento tiene: un resumen de una línea, el estado, quién tiene cada rol, los *criterios de salida* (por ejemplo, "dentro de los SLOs de disponibilidad y latencia durante más de 30 minutos"), la lista de tareas y una línea de tiempo con lo más reciente primero.
- **Un traspaso explícito**: al terminar la jornada, el rol de *incident commander* se entrega de forma explícita ("ahora sos el *incident commander*, ¿de acuerdo?") y no se termina la llamada hasta tener una confirmación firme. El traspaso se comunica a todos.

Los criterios de salida son otra aparición del SLO: el incidente no termina cuando "parece" resuelto, sino cuando el SLI vuelve a su objetivo por un tiempo acordado.

6.3.3 Prioridades durante el incidente

El orden de prioridades es: detener el daño, restablecer el servicio y conservar la evidencia para analizar después la causa.[65] Primero se mitiga, después se entiende. Durante el incidente no se discute la culpa ni el rediseño de la arquitectura; esas preguntas pertenecen al análisis posterior. Además: preparar el proceso de antemano, pedir ayuda si uno se siente desbordado, revisar cada tanto si la estrategia elegida sigue teniendo sentido y practicar el proceso para que no haya que inventarlo en medio del incidente.

La mitigación más frecuente es revertir. En Google, alrededor del 70 % de las caídas se deben a cambios en un sistema en producción.[66] Por eso la primera hipótesis ante un incidente suele ser el último cambio, y la mitigación más rápida, revertirlo. Un caso de Google: un cambio de configuración aplicado a todos los servidores a la vez hizo que casi todos los sistemas expuestos al público empezaran a caerse y reiniciarse en bucle. A los cinco minutos, la persona que había hecho el cambio, al ver las quejas, lo revirtió, y el servicio se recuperó. El cambio no se había considerado riesgoso y por eso no pasó por una prueba *canary* completa; y que alguien notara las quejas tan rápido fue, en parte, suerte.[67]

6.4 Runbooks

Un *runbook* (o *playbook*) documenta de antemano cómo responder. En Google, tener registrados de antemano los pasos de respuesta reduce aproximadamente a la tercera parte el tiempo medio de recuperación respecto de improvisar. Ningún runbook reemplaza a alguien capaz de pensar sobre la marcha, pero los pasos claros ayudan cuando el tiempo apremia, y se complementan con ejercicios en los que el equipo vuelve a jugar un incidente pasado para practicar la respuesta.[68]

Hay también un contrapunto. Mantener un documento vivo que intente contener todos los errores posibles y sus soluciones es un juego inútil y peligroso: la documentación envejece rápido, y una documentación equivocada puede ser más peligrosa que ninguna. Los sistemas distribuidos rara vez fallan dos veces de la misma manera. Lo que sí tiene sentido es una documentación breve de cada servicio: qué equipo es dueño, cómo contactar a la guardia, a quién escalar, de qué servicios depende y cuáles dependen de él, y enlaces a consultas y *dashboards* útiles.[69]

[65] Beyer et al. (2016), cap. 14, recuadro *Best Practices for Incident Management*.
[66] Beyer et al. (2016), cap. 1.
[67] Beyer et al. (2016), cap. 13, *Change-Induced Emergency*.
[68] Beyer et al. (2016), caps. 1 y 15.
[69] Majors et al. (2022), cap. 8, recuadro *On the Topic of Runbooks*.

Las dos posturas son compatibles si el runbook se mantiene breve: una ficha del servicio más una entrada por cada alerta que responda qué significa, cómo confirmarla, qué mitigaciones son seguras y a quién escalar. Para el generador:

> **Servicio:** generador de contraseñas. **Dueño:** equipo X. **Guardia:** canal y rotación. **Escalamiento:** dueño de cada servicio hijo.
>
> **Depende de:** *mayúsculas*, *minúsculas* (que depende de *dígitos*), *dígitos*, *especiales*. **Dashboards:** SLIs, latencia por dependencia, estado de los *breakers*.
>
> **Alerta "burn rate alto, 1 h".** Significa que se consumió al menos el 2 % del presupuesto mensual en la última hora. Confirmar en el *dashboard* de SLIs y mirar latencia y errores por dependencia y el estado de los *breakers*. Mitigaciones seguras: revertir el último despliegue o cambio de configuración; no deshabilitar los *timeouts*; no activar un *fallback* sin especiales (viola la política de contraseñas). Escalar al dueño de la dependencia identificada.

7 Aprender: postmortems

7.1 Qué es y cuándo se escribe

Un postmortem es un registro escrito de un incidente: su impacto, las acciones tomadas para mitigarlo o resolverlo, sus causas y las acciones de seguimiento para evitar que se repita. Su objetivo es que el incidente quede documentado, que sus causas se entiendan bien y, sobre todo, que se tomen acciones preventivas eficaces.[70] Escribir un postmortem no es un castigo; es una oportunidad de aprendizaje para toda la organización.

Escribirlo cuesta tiempo, de modo que conviene fijar de antemano cuándo es obligatorio. Los criterios habituales son: caída o degradación visible para los usuarios por encima de cierto umbral; pérdida de datos de cualquier tipo; intervención de la guardia (revertir una versión, desviar tráfico); un tiempo de resolución por encima de cierto umbral; y una falla del monitoreo, que suele significar que el incidente se descubrió a mano. Cualquier interesado puede, además, pedir un postmortem.[71] Con SLOs, la política de error budget puede agregar uno más, como en el ejemplo de la sección 3.

7.2 Cómo se escribe

7.2.1 Contenido

Un postmortem completo tiene esta estructura:[72]

[70] Beyer et al. (2016), cap. 15.
[71] Beyer et al. (2016), cap. 15.
[72] Beyer et al. (2016), apéndice D.

Sección	Contenido
Resumen	Una o dos líneas: qué pasó y cuánto duró.
Impacto	El efecto sobre usuarios, ingresos, etc., cuantificado; con SLOs, también el error budget consumido.
Causas	Las circunstancias en que ocurrió el incidente; las condiciones que lo hicieron posible.
Disparador	El evento que lo inició.
Resolución	Cómo se mitigó y cómo se resolvió.
Detección	Cómo se enteró el equipo (idealmente, por una alerta y no por un cliente).
Acciones	Cada una con tipo, dueño y seguimiento.
Lecciones	Qué salió bien, qué salió mal y dónde se tuvo suerte.
Línea de tiempo	El "guion" del incidente, empezando por la del documento vivo.
Información de soporte	Enlaces a *dashboards*, logs, gráficos, registros del canal.

La sección "dónde se tuvo suerte" merece atención: registra las cosas que salieron bien sin que nadie lo hubiera garantizado, por ejemplo, que justo estuviera disponible la única persona que conocía un sistema. La próxima vez esa suerte puede no estar.

7.2.2 Blameless

Los postmortems deben ser sin culpables (*blameless*). Para que lo sean, deben concentrarse en identificar las causas que contribuyeron al incidente sin acusar a ninguna persona ni a ningún equipo de haber actuado mal. Un postmortem *blameless* supone que todos los involucrados tuvieron buenas intenciones e hicieron lo correcto con la información que tenían. Si prevalece una cultura de señalar y avergonzar, las personas dejan de contar los problemas por miedo al castigo, y los problemas se esconden.[73]

Esta cultura viene de la salud y la aviación, industrias en las que los errores pueden ser fatales y en las que cada error se ve como una oportunidad de fortalecer el sistema. Cuando el postmortem deja de repartir culpas y pasa a investigar por qué una persona o un equipo tenía información incompleta o incorrecta, se pueden armar planes de prevención eficaces: no se puede "arreglar" a las personas, pero sí los sistemas y los procesos que las ayudan a decidir bien.[74]

Blameless no significa ausencia de responsabilidad ni de crítica. Un postmortem sin culpables igual señala dónde y cómo mejorar. Compárense dos formas de decir lo mismo: "¡Hay que reescribir todo ese *backend*! Se rompe todas las semanas y si me vuelven a llamar lo reescribo yo" frente a "Reescribir el *backend* podría evitar estas alertas; además, su manual de mantenimiento es muy largo y difícil de aprender, y las próximas guardias lo van a agradecer". Ambas proponen la misma acción; solo la segunda la apoya en el sistema.

La pregunta útil. Para analizar una decisión que resultó mal, la pregunta no es "¿por qué se equivocó?" sino "¿qué información, qué presiones y qué herramientas hicieron que esa decisión pareciera razonable en ese momento?". Después de un incidente, todo parece obvio: quien conoce el resultado ve con claridad un camino que, para quien estaba en el momento, no era evidente. Ese sesgo retrospectivo (*hindsight bias*) hace parecer negligentes decisiones que eran razonables con la información disponible.[75] La respuesta a esa pregunta señala mejoras posibles en el sistema; la culpa no señala

[73] Beyer et al. (2016), cap. 15.
[74] Beyer et al. (2016), cap. 15.
[75] Cook (2000), punto 8.

ninguna.

7.2.3 Disparador, causas y factores contribuyentes

Un buen postmortem separa el *disparador* de las *condiciones* que hicieron posible el incidente. El disparador es el evento que lo inició; las condiciones son los defectos y las decisiones previas que estaban ahí, sin causar problemas, esperando ese evento. La distinción importa porque un incidente rara vez tiene una sola causa:[76]

- un sistema que funciona tiene varias defensas, y para que se caiga suelen tener que fallar varias a la vez;
- todo sistema grande tiene en cada momento defectos latentes que no se manifiestan hasta que algo los activa;
- por eso, buscar "la" causa raíz (*root cause*) lleva a corregir un solo eslabón y dejar los demás.

Un incidente ficticio en el generador sirve de ejemplo. Un despliegue cambia la biblioteca HTTP del servicio y, con ella, el *timeout* hacia el servicio de *especiales*, que pasa a ser el valor por defecto: esperar indefinidamente. Días después, *especiales* sufre pausas largas de recolección de basura; los hilos del generador quedan bloqueados, el pool se agota y el generador deja de responder. La alerta de burn rate suena a los dos minutos, pero la mitigación tarda 25 minutos porque el cambio de biblioteca venía junto con otros y revertirlo no era trivial. La lista de factores es más útil que cualquier "causa raíz":

- el disparador: las pausas de *especiales*;
- una condición latente: un *timeout* que dependía del valor por defecto de una biblioteca (la clase 11 advirtió que muchas esperan para siempre);
- una falla de validación: el cambio no se consideró riesgoso y no tuvo *canary*; ningún experimento de latencia sobre *especiales* corría de forma periódica;
- una falla de diagnóstico: el *dashboard* no mostraba la latencia por dependencia;
- una falla de mitigación: un despliegue con varios cambios juntos, difícil de revertir.

Cada factor sugiere una acción distinta. Quedarse con "la causa fue la pausa de *especiales*" no sugiere ninguna útil.

7.2.4 Acciones que cambian el sistema

Un postmortem vale lo que valen sus acciones. Para que sirvan, las acciones deben cumplir estos criterios:[77]

- **Un dueño por acción**, y un dueño del postmortem responsable del seguimiento. Las acciones sin dueño rara vez se cumplen.
- **Registradas** en el sistema de seguimiento de errores del equipo, con su prioridad.
- **Verificables**: con un estado final claro, sin verbos vagos como "mejorar".
- **Prevenir, no solo mitigar**: incluir acciones que eviten la repetición, además de las que reducen el daño. Cambiar sistemas y procesos automatizados es más confiable que pedir a las personas que cambien su comportamiento.

[76] Cook (2000), puntos 3, 4, 7 y 16.
[77] Beyer et al. (2018), cap. 10.

- **Clasificadas por tipo**: investigar, prevenir, mitigar, reparar, detectar.

Por contraste, son malas acciones "tener más cuidado al desplegar" (depende de cambiar a las personas y no se puede verificar), "mejorar el monitoreo" (vaga) o una lista sin dueños ni fechas. Hay también una advertencia en la otra dirección: las acciones propuestas en caliente, apenas terminado el incidente, suelen resultar demasiado extremas o costosas, y conviene revisarlas con calma.[78] En el ejemplo del generador, buenas acciones serían: "el *timeout* hacia cada dependencia es obligatorio en la configuración y el despliegue falla si falta" (prevenir); "experimento semanal automatizado de latencia sobre cada dependencia" (detectar); "panel de latencia por dependencia en el *dashboard* de guardia" (detectar); "despliegues de un solo cambio para las bibliotecas de red" (mitigar).

7.2.5 Un postmortem malo y uno bueno

Las diferencias entre un postmortem malo y uno bueno sobre el mismo incidente suelen ser estas:[79]

Malo	Bueno
Usa términos internos sin explicarlos.	Lo entiende alguien de otro equipo.
Impacto vago ("afectó a muchos usuarios").	Impacto medido, con su fuente.
Acciones vagas, sin dueño, todas con la misma prioridad.	Acciones con dueño, prioridad y estado final verificable.
Señala a personas y usa un tono dramático.	Describe qué falló en el sistema, no quién.
Se publica meses después y solo dentro del equipo.	Circula en pocos días y se comparte ampliamente.

7.3 Revisar y compartir

Un postmortem que nadie revisa es como si no existiera.[80] Conviene que el borrador lo revise alguien con experiencia, con preguntas como: ¿se recolectaron los datos clave del incidente?, ¿está completa la evaluación del impacto?, ¿la causa es suficientemente profunda?, ¿el plan de acción es adecuado y las correcciones tienen la prioridad correcta?, ¿se compartió el resultado con los interesados? Después se comparte lo más ampliamente posible, sin datos que identifiquen usuarios, y se archiva en un repositorio de incidentes donde otros puedan aprender.

La cultura se cultiva con prácticas concretas: compartir los postmortems interesantes, leerlos en grupo, practicar con incidentes pasados y reconocer en público a quien maneja bien un incidente, aunque lo haya provocado. En Google, un ingeniero cuyo cambio dejó caído un servicio crítico durante cuatro minutos recibió un reconocimiento público, porque fueron solo cuatro gracias a que revirtió el cambio de inmediato.[81] También hay señales de que la cultura está fallando: que se eviten los postmortems, que los directivos usen lenguaje que culpa, que no haya tiempo para escribirlos bien, y sobre todo que los incidentes se repitan, lo que suele indicar acciones que se cierran tarde o parches que no resuelven el problema.[82]

> El valor de un incidente no está solo en restaurar el servicio, sino en convertirlo en capacidad futura. Eso exige un postmortem sin culpables, que separe el disparador de las condiciones que lo hicieron posible, con acciones verificables, con dueño, que cambien el sistema y no le pidan a

[78] Beyer et al. (2016), apéndice D, nota 3.
[79] Beyer et al. (2018), cap. 10.
[80] Beyer et al. (2016), cap. 15.
[81] Beyer et al. (2016), cap. 15.
[82] Beyer et al. (2018), cap. 10.

las personas "tener más cuidado".

8 Modelo del curso

Expresamos el problema de la confiabilidad operativa mediante el framework del curso:

$$D = (P, F, G, S, T)$$

- P **(Problema):** Operar un sistema distribuido con un nivel de servicio explícito, verificar que se cumple bajo falla y aprender cuando no se cumple.
- F **(Factores/Restricciones):** Fallas parciales inevitables, propiedades emergentes que no se ven en los componentes, cambios frecuentes, mediciones imperfectas, poco tráfico en algunos servicios, personas que deciden bajo presión con información incompleta.
- G **(Garantías/Objetivos):** SLOs medidos desde el usuario, alertas proporcionales a la gravedad, hipótesis de resiliencia validadas con evidencia, respuesta coordinada y acciones que reducen la repetición.
- S **(Solución):** SLIs como cociente de eventos buenos, SLOs con ventana deslizante, error budget con política acordada, alertas de burn rate con varias ventanas, experimentos bajo falla con *blast radius* y criterio de aborto, roles de incidente, runbooks breves, postmortems sin culpables con acciones verificables.
- T **(Trade-offs/Costos):** Cada nueve cuesta más que el anterior; los experimentos consumen error budget y pueden convertirse en incidentes; las alertas cambian precisión por tiempo de detección; la documentación envejece; escribir y revisar postmortems lleva tiempo.

9 Conclusión

La confiabilidad operativa no se gestiona con intuiciones. Se define qué significa "funcionar suficientemente bien" con un SLI medido desde el usuario y un SLO con su ventana; el error budget convierte ese objetivo en una regla para decidir cuánto arriesgar; el burn rate lo transforma en alertas que avisan con urgencia proporcional al daño. Sobre esa base, los experimentos bajo falla validan que los mecanismos de la clase 11 hacen lo que se supone, con el SLI como estado estable y un costo calculado en presupuesto. Cuando algo falla igual —y va a fallar—, un proceso con roles claros reduce el daño, y un postmortem sin culpables transforma el incidente en acciones que cambian el sistema.

Las piezas dependen unas de otras: sin SLI no hay *steady state* para experimentar ni criterio para declarar resuelto un incidente; sin error budget no hay forma de decidir si un experimento se puede correr ni de justificar frenar los despliegues; sin postmortem, los incidentes se repiten.

Errores de razonamiento frecuentes

- fijar el SLO en 100 %, o a partir de lo que el sistema logra hoy;
- elegir SLIs que miden la infraestructura (CPU, memoria) en lugar de la experiencia del usuario;
- contar como "bueno" todo código 200, aunque la respuesta sea incorrecta o esté degradada;
- usar el promedio de latencia en lugar de una proporción de pedidos por debajo de un umbral;

- alertar recién cuando el presupuesto se agotó, o con un umbral fijo que no escala con la gravedad;
- creer que Chaos Engineering es romper cosas al azar, o experimentar sin hipótesis, sin *steady state* o sin criterio de aborto;
- experimentar en producción sin calcular cuánto error budget se va a consumir;
- buscar una única causa raíz, o reducir el incidente a un "error humano";
- cerrar un postmortem con acciones como "tener más cuidado", sin dueño ni fecha.

Una organización madura no se define por no tener incidentes, sino por medir con objetivos explícitos, validar sus supuestos antes de que la realidad lo haga y aprender de manera sistemática cuando algo falla. La confiabilidad es una propiedad del sistema y también de la organización que lo opera.

10 Aplicación al trabajo práctico

El trabajo práctico (Asteria Online) pide construir tuplas $D = (P, F, G, S, T)$ en tres entregas, a nivel de mecanismo y sin implementar. No pide SLOs, experimentos ni postmortems como entregables, pero varias ideas de esta clase sirven para escribir mejor las soluciones (S) y los trade-offs (T).

La garantía no lleva números; la solución, sí. El enunciado pide que las garantías no tengan promesas absolutas ("nunca", "siempre") ni números concretos ("en menos de 2 segundos"): esos compromisos son un atributo de la solución, no del interés que se protege. Es la misma separación de esta clase. La garantía describe qué le importa al jugador, como la especificación de un SLI (sección 2.2). El SLO, en cambio, es el grado con que una solución concreta sostiene esa garantía, y por eso pertenece a S y a T. Por ejemplo, para la garantía "el jugador no debe perder el oro pagado por un combate que no llegó a resolverse", una solución podría declarar como SLI la proporción de entradas pagadas a combates no resueltos en las que el oro volvió al jugador, y un SLO sobre ese SLI. Otra solución para la misma garantía podría declarar un SLO distinto.

La brecha parcial, cuantificada. En la Entrega 3, cuando una solución sostiene su garantía solo de forma parcial, hay que explicitar la brecha: bajo qué condiciones el interés queda expuesto. El error budget da una forma de medirla y de comparar dos soluciones con el mismo criterio. Por ejemplo, con un SLO de 99,9 % de disponibilidad de las instancias de combate en 30 días (43,2 minutos de presupuesto):

- una solución A, que ante la caída de un datacenter reubica las instancias en otro en 15 minutos, y
- una solución B, que mantiene réplicas listas y las reubica en 1 minuto.

Si en el datacenter caído corre un tercio de las instancias, la caída con la solución A equivale a 5 minutos de caída total (un tercio de 15) y consume el 12 % del presupuesto; con la solución B, unos 20 segundos y menos del 1 %. Los números son ilustrativos, pero muestran la forma de la comparación objetiva que pide la Entrega 3: el mismo SLI, el mismo presupuesto y cuánto consume cada solución ante la misma falla, frente al costo de cada una.

Un SLI por subsistema y por tipo de jugador. El enunciado prevé fallas parciales (la caída del directorio de zonas, de los combates o de los pagos) y exige que, ante picos de demanda, el juego priorice a quienes ya están jugando por sobre quienes intentan ingresar. Un SLI global esconde las dos

cosas: una caída de los pagos se diluye en el promedio de todo el juego. Conviene pensar un SLI por subsistema, y separar el de los jugadores que ya están en una partida del de los que intentan ingresar, con objetivos distintos. El *load shedding* de la clase 11 es, en esos términos, una decisión explícita de gastar el presupuesto del ingreso para proteger el de los jugadores que ya están jugando.

Cómo se validaría una solución. Aunque no se implemente, una solución se defiende mejor si se puede decir cómo se comprobaría que funciona. Los campos del *template* de experimento (sección 5.3.5) sirven para eso: qué falla se inyectaría (por ejemplo, aislar Zurich de los otros dos datacenters), qué se observaría (el SLI del directorio de zonas en cada datacenter), qué resultado sostendría la decisión CAP elegida y qué resultado la refutaría. Pensarlo así suele revelar trade-offs que no aparecen al describir la solución en abstracto.

11 Bibliografía guiada y ruta de profundización

Para profundizar se recomienda la siguiente ruta, con qué buscar en cada lectura:

- *Site Reliability Engineering*, cap. 3: por qué el 100 % no es el objetivo y cómo el error budget alinea a desarrollo y operaciones. Cap. 4: la terminología, cómo elegir indicadores y objetivos, y el caso Chubby. Cap. 13: los tres casos de emergencia; buscar qué salió bien, qué salió mal y qué se aprendió en cada uno. Cap. 14: comparar el incidente no gestionado con el gestionado. Cap. 15: los disparadores de postmortem y la cultura *blameless*. Apéndices C y D: un ejemplo completo de documento vivo de incidente y otro de postmortem, útiles como *templates*.
- *The Site Reliability Workbook*, cap. 2: la diferencia entre especificación e implementación del SLI y la política de error budget. Cap. 5: rehacer las cuentas de los seis enfoques de alerta. Cap. 10: comparar el postmortem malo con el bueno.
- *Release It!*, cap. 17: los prerrequisitos, el diseño del experimento y los tipos de inyección; buscar qué debilidad encuentra cada uno.
- *Observability Engineering*, caps. 12 y 13: por qué alertar sobre síntomas y cómo se calcula una alerta predictiva de consumo del presupuesto. Cap. 8, recuadro sobre runbooks: qué documentación sí vale la pena mantener.
- *Principles of Chaos Engineering*: la definición y los principios, en una página.
- Cook, *How Complex Systems Fail*: dieciocho puntos breves; leerlos pensando en el último incidente que se haya vivido.

Lecturas complementarias, para quien quiera ir más lejos: Allspaw (2012) sobre postmortems sin culpables y cultura justa, que el libro de SRE cita en su capítulo 15; Basiri et al. (2016), la presentación de Chaos Engineering por el equipo de Netflix; Hidalgo (2020), un libro completo sobre SLOs, que *Observability Engineering* recomienda para políticas de error budget; y Dekker (2011), sobre la deriva hacia la falla.

Durante la lectura, registre: qué se mide y desde dónde, qué decisión habilita cada número, qué supuesto valida cada experimento, y qué acción del postmortem cambia el sistema y cuál solo pide a las personas que cambien.

Referencias canónicas

1. **Beyer, B., Jones, C., Petoff, J. y Murphy, N. R.** (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media. (Capítulos 1, sección sobre *playbooks*; 3,

Embracing Risk; 4, *Service Level Objectives*; 13, *Emergency Response*; 14, *Managing Incidents*; 15, *Postmortem Culture: Learning from Failure*; apéndices C, *Example Incident State Document*, y D, *Example Postmortem*).

2. **Beyer, B., Murphy, N. R., Rensin, D. K., Kawahara, K. y Thorne, S.** (2018). *The Site Reliability Workbook: Practical Ways to Implement SRE*. O'Reilly Media. (Capítulos 2, *Implementing SLOs*; 5, *Alerting on SLOs*; 10, *Postmortem Culture: Learning from Failure*). Disponible en `https://sre.google/workbook/table-of-contents/`.
3. **Nygard, M. T.** (2018). *Release It! Design and Deploy Production-Ready Software* (2.ª ed.). Pragmatic Bookshelf. (Capítulo 17, *Chaos Engineering*).
4. **Majors, C., Fong-Jones, L. y Miranda, G.** (2022). *Observability Engineering: Achieving Production Excellence*. O'Reilly Media. (Capítulos 12, *Using Service-Level Objectives for Reliability*, y 13, *Acting on and Debugging SLO-Based Alerts*; capítulo 8, recuadro *On the Topic of Runbooks*).
5. **Principles of Chaos Engineering** (2019). `https://principlesofchaos.org/` (última actualización: marzo de 2019).
6. **Cook, R. I.** (2000). *How Complex Systems Fail*. Cognitive Technologies Laboratory, University of Chicago. `https://how.complexsystems.fail/`

Lecturas complementarias

1. **Allspaw, J.** (2012). *Blameless PostMortems and a Just Culture*. Etsy, Code as Craft.
2. **Basiri, A. et al.** (2016). Chaos Engineering. *IEEE Software*, 33(3).
3. **Hidalgo, A.** (2020). *Implementing Service Level Objectives*. O'Reilly Media.
4. **Dekker, S.** (2011). *Drift into Failure*. CRC Press.